Lab No: 05

HIKMAT ULLAH JAN

Electrical COMMUNICATION

Semester 2nd

Section B

Registration BF25NWELE0662

Lab Title: **while Loop in C++**

Learning objectives:

The objectives of this lab are:

- **To understand the working of the while loop in C++.**
- **To learn how to implement a do...while loop.**
- **To understand the concept of a nested while loop.** • **To use loops for solving programming problems such as printing numbers and calculating sums.**

Loop

Loops are used to repeat a block of code. Being able to have your program repeatedly execute a block of code is one of the most basic but useful tasks in programming -- many programs or websites that produce extremely complex output are really only executing a single task many times. Now, think about what this means: a loop lets you write a very simple statement to produce a significantly greater result simply by repetition.

While Loop

The most basic loop in C++ is the while loop. A while statement is like a repeating if statement. Like an If statement, if the test condition is true: the statements get executed. The difference is that after the statements have been executed, the test condition is checked again. If it is still true the statements get executed again. This cycle repeats until the test condition evaluates too false.

A **while loop** is a control flow statement that allows code to be executed repeatedly based on a given condition. The *while* consists of a block of code and a condition. The condition is first evaluated

and if the condition is true the code within the block is then executed. This repeats until the condition becomes false

Syntax

```
while (expression)
{
Single statement or
Block of statements; }
```

The expression can be any combination of Boolean statements that are legal.

- **Example:**

This program prints numbers from **1 to 5** using a while loop.

```
#include <iostream>
using namespace std;
int main() {
int i = 1;                    // initialize variable
while (i <= 5) {              // condition
cout << i << endl;
i++;                          // increment
}
return 0;
}
```

Nested While Loop

Definition

A **nested loop** means placing **one loop inside another loop**.

In nested loops:

- the **outer loop controls rows**
- the **inner loop controls columns**

Nested loops are commonly used for:

- patterns
- matrices
- tables

Syntax

```
while(condition1)
    while(condition2)
        // statements
```

Example

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;
    while(i <= 3)
    {
        int j = 1;

        while(j <= 3)
        {
            cout << "* ";
            j++;
        }
        cout << endl;
        i++;
}
    return 0;
}
```

Output

```
* * *
* * *
* * *
```

3 Do-While Loop

Definition

A **do-while loop** is similar to a while loop, but the condition is checked **after the execution of the loop body**.

Therefore, the loop **executes at least one time** even if the condition is false.

Syntax

```
do
{
    // statements
}
while(condition);
```

⚠ A **semicolon (;)** is required after the condition.

Example

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;

    do
    {
        cout << i << endl;
        i++;
    }
    while(i <= 5);

    return 0;
}
```

Output:1 2 3 4 5

Lab Tasks

Task 1:

1. **Write a C++ program that uses a while loop to display numbers from 1 to 10**
2. **each on a separate line. with comments**

```
#include <iostream>
```

```
using namespace std;
int main() {
int i = 1;                    // Step 1: Initialize the counter variable
while (i <= 10) {             // Step 2: Start the while loop (runs while condition is true)
  cout << i << endl;          // Step 3: Print the current value of
   i++;                       // Step 4: Increment the counter (increase i by 1)
   } return 0;}               // Step 5: End of program
```

Task 2: Using do-while Loop

Repeat the **first task** using a **do-while loop**.

```
#include <iostream>
using namespace std;
int main() {
int i = 1;                    // Step 1: Initialize the counter variable
do {                          // Step 2: Start the do-while loop
      cout << i << endl;      // Step 3: Print the current value of i
      i++;                    // Step 4: Increment the counter
   } while (i <= 10);         // Step 5: Condition is checked after execution
return 0;                     // End of program
}
```

Task 3: Sum Using While Loop

Write a C++ program that:

1. Prompts the user to enter a **positive integer**.
2. Uses a **while loop** to calculate the **sum from 1 to that number**.
3. Displays the calculated **sum**.

```
#include <iostream>
using namespace std;

int main() {
int num, i = 1;
int sum = 0;
cout << "Enter a positive integer: ";          // Step 1: Prompt the user to enter a positive integer
cin >> num;
while (i <= num) {                             // Step 2: Use while loop to calculate sum from 1 to num
 sum = sum + i;                                // Add current value of i to sum
 i++;                                          // Increment i
   }
cout << "The sum from 1 to " << num << " is: " << sum << endl;
                                               // Step 3: Display the result

   return 0;
}
```